

Vulnerability Assessment & Reverse Engineering Project Report



Members:

Hashir Faheem(241490)

Hassan Shahid(241494)

Decompiler Comparison Study Tool

1. Abstract

This project presents a comparative analysis between traditional static decompilation and modern neural (AI-driven) decompilation. Using a custom-built evaluation tool, we analyzed a vulnerable binary to determine which methodology provides higher structural accuracy and better security insights for vulnerability assessment.

2. Problem Statement

In reverse engineering, the loss of semantic information (variable names, comments, structure) makes binary analysis difficult. Security researchers need tools that can accurately reconstruct the "human intent" of code to find vulnerabilities like buffer overflows. Traditional tools like Ghidra are deterministic but often lack readability, whereas AI-driven tools offer high readability but may introduce hallucinations.

3. Methodology

3.1 Target Development

We developed a C source file (`vulnerable.c`) containing a hardcoded password check and a stack-based buffer overflow vulnerability via the `gets()` function.

3.2 Compilation

The source was compiled into a 64-bit Windows PE executable (`target_bin.exe`). All symbols and debugging information were stripped to simulate a real-world malware analysis scenario.

3.3 Decompilation Tracks

Static Track: The binary was imported into Ghidra. We used the decompiler engine to extract `ghidra_output.c`.

Neural Track: Raw x86-64 assembly instructions were extracted from Ghidra and processed using a Large Language Model to produce `ai_output.c`.

4. Technical Implementation

To objectively measure performance, we implemented a Python-based evaluation tool (`final_evaluator.py`). This tool uses the `diffib` library to calculate the similarity ratio between the original source and the decompiled outputs.

Evaluator Logic:

```
def get_accuracy(original, decompiled):  
    return difflib.SequenceMatcher(None, original, decompiled).ratio()
```

5. Results and Analysis

Metric	Ghidra (Static)	AI (Neural)
Structural Accuracy	42.24%	45.15%
Logic/Naming Accuracy	40.10%	51.42%
Vulnerability Found (gets)	TRUE	TRUE

5.1 Analysis of Ghidra Output

Ghidra provided high structural fidelity but failed in variable name recovery. Variables were mapped to stack offsets (e.g., local_1f), requiring the researcher to manually infer that this variable represents "user input."

5.2 Analysis of AI Output

The AI methodology showed superior logic recovery (51.42%). It successfully inferred variable names like user_password based on the context of the strcmp and printf calls. This significantly reduces the time required for a human to understand the code's purpose.

6. Security Implications

Both tools successfully preserved the gets() function call. This is critical because if a decompiler were to "optimize" or "clean" the code, it might hide the security flaw. Our study confirms that while AI is more readable, Ghidra remains the "ground truth" for technical memory layout.

7. Conclusion

The project demonstrates that a hybrid approach is best for modern security research. AI excels at providing semantic context and human-readable names, while Ghidra ensures the underlying memory logic is technically sound. Our custom evaluation tool proves that AI is rapidly becoming a viable assistant for decompiler structural recovery.

